



Mastering DAX

Multiple Filters

with

CALCULATE ()

Function



Azman Rana
Expert Data Analytics

Understanding DAX's New Feature: Multiple Column Filters

This feature allows you to apply filters across multiple columns in a single DAX function, making calculations more efficient, precise, and easier to write.



Code Highlight:

Before this update, filtering on multiple columns required more complex approaches like nested `FILTER` or manually combining conditions. Now, it's as simple as listing your conditions!

dax

```
CALCULATE(SUM(Sales[Amount]), Sales[Color] = "Red",  
Sales[Brand] = "Contoso")
```

Pro Tip:

Focus on small, precise filters instead of filtering entire tables for better query performance.



Azman Rana
Expert Data Analytics

01



What's New?

DAX now supports multiple column filters in CALCULATE! 🎉

This is a huge improvement for anyone working with large datasets where filtering across multiple columns was once cumbersome.

What's the benefit?

Before, complex filtering conditions across columns made your code harder to read and troubleshoot. With this update, DAX can handle:

- Filters on multiple columns in the same table with ease.
- Streamlined code for faster calculations.



Azman Rana
Expert Data Analytics

Code Example:

Here's a simple example:

```
```dax
CALCULATE(SUM(Sales[Amount]),
 Sales[Color] = "Red", Sales[Brand] = "Contoso")
```
```

Before the update: This would throw an error or need workarounds, especially if you wanted multiple column filters from different tables.



Azman Rana
Expert Data Analytics

02



Why is it Important?

Previously, filtering on multiple columns was a challenge. You had to use `FILTER` functions or complicated logic.

For example, to calculate sales of red products from the brand Contoso, you would write:

```
``dax  
FILTER(Sales, Sales[Color] = "Red" && Sales[Brand]  
= "Contoso")  
``
```



Azman Rana
Expert Data Analytics

Using Multiple Filters

Why is this bad?

- It's less efficient.
- It's harder to read.
- It's prone to mistakes.



With the new update, DAX does this natively using the `CALCULATE` function.

Code Example:

```
dax
CALCULATE(SUM(Sales[Amount]),
  Sales[Color] = "Red", Sales[Brand] = "Contoso")
```

Helpful Tip:

Always prioritize using `CALCULATE` with simple column filters over `FILTER` functions where possible. It's more efficient!



Azman Rana
Expert Data Analytics

03



How **CALCULATE()** Works

The `CALCULATE` function modifies the filter context of a measure by applying filters to the dataset. This allows you to control how calculations are performed, depending on the filtered data.

Let's start with a simple example:
I created a measure `Red Sales` that filters sales where the color is red.

```
dax
Red Sales = CALCULATE(SUM(Sales[Amount]),
Sales[Color] = "Red")
```

Helpful Tip:

`CALCULATE` works by adjusting the filter context, meaning you can control how specific calculations should behave based on different filters.

04



Multiple Filters Example

We now want to calculate sales for:

- Products with the color red
- Products from the brand "Contoso"

The old way of handling this was inefficient, as we had to either manually manage conditions or use `FILTER` functions.

```
dax
CALCULATE(SUM(Sales[Amount]),
  Sales[Color] = "Red",
  Sales[Brand] = "Contoso")
```



Azman Rana
Expert Data Analytics

05



Writing the Query

Writing the Query

Let's break down how we can use DAX to apply multiple filters in a simple query.

Here's the scenario:

We want to filter sales where either the product color is red or the product brand is Contoso.

Old Way:

You'd use `FILTER` or some workaround:

```
```dax
FILTER(Sales, Sales[Color] = "Red" || Sales[Brand] = "Contoso")
```
```

This method works but can get slow with larger datasets and complex queries.



Azman Rana
Expert Data Analytics

06

New Way



New Way:

Using multiple filters in `CALCULATE`:

```
```dax
CALCULATE(SUM(Sales[Amount]),
 OR(Sales[Color] = "Red", Sales[Brand] = "Contoso"))
```
```

Key Takeaway:

The new `OR` logic in `CALCULATE` simplifies the process of applying multiple conditions, improving both readability and performance.



Azman Rana
Expert Data Analytics

07



CALCULATE () with SELECTED VALUE

What Changed?

Before this update, DAX struggled when you tried to apply multiple filters across columns. These conditions were translated into table filters, and mixing multiple column conditions often resulted in errors.

Now, DAX can handle multiple filters more efficiently without needing to filter entire tables.

Helpful Code Snippet:

Here's how DAX processes multiple filters now:

```
```dax
CALCULATE(SUM(Sales[Amount]),
 Sales[Color] = "Red",
 Sales[Brand] = "Contoso")
```
```

Why is this better?

DAX now applies the filters at the column level, which speeds up calculations and makes the code cleaner.



Azman Rana
Expert Data Analytics

08



Old Approach vs New Approach

Old Approach vs New Approach

Here's a quick comparison of how things have improved:

Old Approach:

```
```dax
FILTER(Sales, Sales[Color] = "Red" || Sales[Brand] = "Contoso")
```
```

- Slower performance due to filtering the entire table.
- Harder to maintain when the logic gets more complex.

New Approach:

```
```dax
CALCULATE(SUM(Sales[Amount]),
 Sales[Color] = "Red",
 Sales[Brand] = "Contoso")
```
```

- Faster, because we're only filtering the specific columns.
- Easier to read and maintain.





09

Optimization with ALL function

Optimization with ALL Function

The `ALL()` function is one of DAX's most powerful features when it comes to controlling filters. By using `ALL()`, you can remove existing filters and create a clean filter context.

Let's say you want to calculate total sales without being affected by any filters already applied to the Color column:

```
```dax
CALCULATE(SUM(Sales[Amount]), ALL(Sales[Color]))
```
```

This removes any color-based filters from the current context, letting you focus on a new filter (e.g., Brand).

Helpful Tip:

Use `ALL()` when you need to override filters in specific columns but be cautious—removing too many filters can lead to unintended results.





10

Avoiding Over-Filtration

Avoiding Over-Filtration

It's tempting to filter entire tables when building complex queries, but this can drastically slow down performance. Instead, focus on filtering specific columns to minimize computation time and increase accuracy.

Bad Practice:

```
```dax
FILTER(Product, Product[Color] = "Red")
```
```

This filters the entire 'Product' table, which can slow down your report.

Good Practice:

```
```dax
CALCULATE(SUM(Sales[Amount]), Sales[Color] = "Red")
```
```

By filtering just the column you need, you keep your query lean and fast.

Helpful Tip:

Filtering entire tables should be avoided unless absolutely necessary. Stick to column-level filters to ensure optimal performance.



11



Best Practices in Action

Best Practices in Action

One of the biggest advantages of the new DAX feature is the ability to follow best practices more easily. Instead of writing complicated filters, you can target just the necessary columns.

Example:

```
dax
CALCULATE(SUM(Sales[Amount]),
  ALL(Sales[Color]),
  Sales[Brand] = "Contoso")
```

Best Practice Tip:

Focus on filtering the smallest number of columns needed to answer your query. This makes your calculations faster and ensures that you are not over-complicating your logic.

:



Azman Rana
Expert Data Analytics



12

Filter Multiple Columns Effectively

Filter Multiple Columns Effectively

The new DAX update allows you to filter multiple columns at once, but there are some key limitations to be aware of.

For example, filtering two columns from the same table works perfectly:

```
```dax
CALCULATE(SUM(Sales[Amount]),
 Sales[Color] = "Red",
 Sales[Brand] = "Contoso")
```

However, if you try to filter across multiple tables without any relationships, you'll run into errors.

---

Key Takeaway:

This feature works best when you're filtering columns from the same table. Always double-check your table relationships to avoid issues.



**Azman Rana**  
**Expert Data Analytics**



# 13

## Handling Complex Conditions

### Handling Complex Conditions

For more complex filters, you can combine multiple conditions using logical operators like `AND` and `OR`:

```
```dax
CALCULATE(SUM(Sales[Amount]),
    AND(Sales[Color] = "Red", Sales[Brand] = "Contoso"))
```
```

This ensures that your calculations are based on multiple conditions at once, without any manual `FILTER` functions.

Best Practice Tip:

When writing complex conditions, use parentheses and clear variable names to keep your DAX code readable and easy to debug.



***Azman Rana***  
***Expert Data Analytics***

# 14

## Nested CALCULATE ()

### Limits of the Feature

While DAX's new feature is powerful, it does have limitations. You can't filter columns from different tables unless those tables are related in the data model.

Example of What Won't Work:

```
```dax
CALCULATE(SUM(Sales[Amount]),
    Sales[Color] = "Red",
    Date[Year] = 2020)
```
```

This throws an error because `Sales` and `Date` are different tables. However, there are ways around this.

---

Pro Tip:

If you need to filter across different tables, you should build relationships in your data model or use more advanced techniques like cross joins.



***Azman Rana***  
***Expert Data Analytics***

# 15

## Participate in relevant groups and discussions

### Key Tip: Cross Join

When filtering across multiple tables, one option is to use `CROSSJOIN()` to create a temporary table that combines data from both tables:

```
```dax
CROSSJOIN(ALL(Sales[Color]), ALL(Date[Year]))
```
```

This allows you to apply filters across multiple tables, as long as you build a combined dataset.

---

Code Example:

```
```dax
CALCULATE(SUM(Sales[Amount]),
  FILTER(CROSSJOIN(ALL(Sales[Color]), ALL(Date[Year])),
    Sales[Color] = "Red" && Date[Year] = 2020))
```
```

Pro Tip:

Use cross joins sparingly, as they can impact performance if used on very large datasets.



***Azman Rana***  
***Expert Data Analytics***

# 16

## Keep Filters Modifier

### Keep Filters Modifier

Want to apply new filters without removing existing ones? Use the `KEEPFILTERS()` function. This allows you to merge the current filter context with your new filter:

```
```dax
CALCULATE(SUM(Sales[Amount]),
    KEEPFILTERS(Sales[Color] = "Red"))
```
```

#### What Does This Do?

Instead of replacing the filters that were already applied, DAX keeps them and adds the new condition on top.

---

#### Pro Tip:

Use `KEEPFILTERS()` when you want to maintain previous filters while adding new conditions. It helps you keep a multi-layered filter context intact.



***Azman Rana***  
***Expert Data Analytics***

# 17

## Practical Example: Keep Filters in Action

### Practical Example: Keep Filters in Action

Let's say you want to keep the filters for both product color and brand:

```
```dax
CALCULATE(SUM(Sales[Amount]),
    KEEPFILTERS(Sales[Color] = "Red"),
    KEEPFILTERS(Sales[Brand] = "Contoso"))
```
```

This example ensures that your calculations only consider red Contoso products, but any other filters from your model (e.g., region, year) will still apply.

---

Helpful Tip:

`KEEPFILTERS()` is perfect when you want granular control over which filters stay active and which ones are replaced by new conditions.



**Azman Rana**  
*Expert Data Analytics*

# 18

## Boost Efficiency

### Boost Efficiency

Using `KEEPFILTERS()` ensures that you don't unintentionally remove filters that are important to your overall analysis. It also helps optimize performance by preventing unnecessary recalculations.

Here's a scenario where it helps:

You're calculating sales for Contoso, but you still want the region filter from the original context to apply:

```
```dax
CALCULATE(SUM(Sales[Amount]),
    KEEPFILTERS(Sales[Brand] = "Contoso"))
```
```

Now, the region filter stays active!

---

Pro Tip:

`KEEPFILTERS()` is a great tool when combining multiple filters from different contexts. It keeps everything efficient and organized.



***Azman Rana***  
***Expert Data Analytics***

## Wrapping Up: DAX Just Got Better!

The ability to use multiple column filters in `CALCULATE` is a small but powerful update that makes your DAX code simpler and more efficient. Here's what you've learned:

- You can filter multiple columns in the same table without complicated `FILTER` functions.
- Performance and readability improve dramatically.
- `ALL()` and `KEEPFILTERS()` allow precise control over filter contexts.

---

Code Summary:

```
```dax
CALCULATE(SUM(Sales[Amount]),
    Sales[Color] = "Red",
    Sales[Brand] = "Contoso")
```
```

Start using these features today to optimize your Power BI and DAX models!



***Azman Rana***  
***Expert Data Analytics***

# 20

## Nested CALCULATE ()

In complex scenarios, you might need to nest CALCULATE functions to apply multiple layers of context modifications.

```
AdjustedSales = CALCULATE (
 CALCULATE (SUM(Sales[Amount]), Sales[Year] = 2023),
 Sales[Category] = "A"
)
```



***Azman Rana***  
***Expert Data Analytics***



# DM me “Data” I will Send it to you



***Azman Rana***  
***Expert Data Analytics***